

Технически Университет – София

Факултет “Компютърни системи и технологии”



Дипломна Работа

*Разработване на уеб чат бот за
новинарски сайт*

Дипломант: Окан Сезгин Сеид, фак. н. 121219017

Специалност: Компютърно и софтуерно инженерство

Ръководител: доц. д-р Галя Павлова

София, 2026

Съдържание:

- 1. Увод**
- 2. Въведение в технологиите за чат бот**
 - 2.1 Същност, произход и приложение на чатботовете
 - 2.2 История и развитие на чатбот технологиите
 - 2.3 Видове чатботове
 - 2.4 Технологиите при съвременните чатботове
 - 2.5 Употреба и приложение на чатботовете
 - 2.6 Предимства и недостатъци на чатботовете
 - 2.7 Бъдещо развитие и тенденции на чатбот технологиите
- 3. Проектиране на системата**
- 4. Програмна реализация**
 - 4.1 Scraper
 - 4.2 Backend
 - 4.3 Frontend
- 5. Ръководство за използване**
- 6. Заключение**
- 7. Използвани източници**

I. Увод

Уеб чат ботовете са системи, които позволяват на потребителите на уебсайтове да комуникират в реално време под интерактивна форма с графичен интерфейс. Най-често са вградени в уебсайта, който потребителят посещава, и не се налага допълнителна инсталация или настройки, а само използването на уеб браузър. Въпреки че първият чат бот датира от 60-те години на XX век, същественото развитие на чат ботовете съвпада с това на основните уеб технологии (**HTML**) през 90-те години на 20-ти век. Тенденциите през годините обхващат от прости и обикновени интерфейси с динамични HTML уеб страници през чат ботове, запазени с графични възможности от програмния език Java, до по-модерни такива с JavaScript в днешно време, а много често и със сложни алгоритми, машинно обучение и изкуствен интелект. В основата на работата на чат ботовете стоят предварително зададени правила, данни и логика, с които да симулират естествен разговор с потребителя и неговите намерения. Днес чат ботовете намират голямо приложение в онлайн платформите на редица бизнеси. Те могат да извършват услуги като: отговаряне на FAQ – често задавани въпроси, изпращане на персонализирани известия, интеграция с електронни пощи, интернет плащания и други. [1, 2, 3, 5, 7, 9]

II. Въведение в технологииите за чат бот

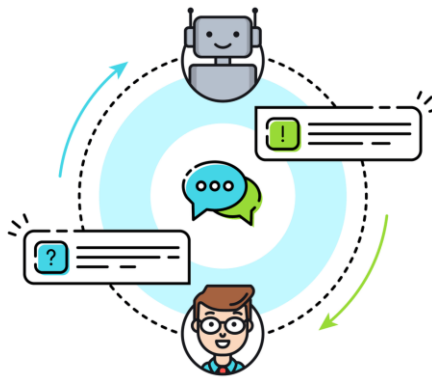
Развитието на дигиталната екосистема налага превръщането на чат ботовете от прости предварително зададени скриптове за автоматични отговори в комплексни софтуерни приложения за симулиране на естествен човешки разговор в реално време.



2.1 Същност, произход и приложение на чатботовете

Чатботовете са софтуерни приложения, създадени с цел да симулират естествен разговор между човек и компютър чрез текстова или гласова комуникация. Основната им функция е да взаимодействат с потребителите, като предоставят актуална информация, отговарят на въпроси, изпълняват задачи и заявки и подпомагат изпълнението на най-различни процеси. Наименованието „**чатбот**“ произлиза от английските думи „chat“ (кратък разговор) и „bot“ (робот), което в буквален превод на български език означава „робот за разговор“. Още се срещат и други имена като chatterbot, talkbot, smartbot и виртуален асистент. Първите технологични разработки в областта на чатботовете се появяват още през 60-те години на XX век. Първият известен чатбот е **ELIZA**, създаден през 1966 г. от Джоузеф Вайзенбаум. Въпреки ограничените си начални възможности, той поставя основите на концепцията за автоматизирана комуникация между човек и машина/компютър. С развитието на компютърните технологии, интернет и изкуствения интелект чатботовете постепенно се превръщат в неизменна част от съвременната дигитална среда. Модерните чатботове не разчитат само на предварително зададени отговори. Те могат да разбират контекста, темата и идеята на разговора, да анализират потребителските намерения и да генерират персонализирани и смислени отговори. Благодарение на това те

намират огромно приложение в електронната търговия, образованието, банковия сектор, здравеопазването, новинарските медии, туристическите и HR агенции и множество други области. [2, 3, 5, 7, 8, 9, 10]



2.2 История и развитие на чатбот технологиите

Развитието на чатботовете преминава през няколко основни етапа. Първоначално те представляват сравнително прости системи, които използват набор от предварително дефинирани правила, логика и шаблонни начини за комуникация. Тези системи са могли да разбират и отговарят само и единствено на ограничен набор от въпроси и не са притежавали способността да разбират естествения език на потребителите. С навлизането на интернет и социалните мрежи започва засилен интерес към автоматизираната онлайн комуникация. Платформи като Facebook Messenger, Telegram, Viber и WhatsApp създават подходяща среда за развитието на чатботове, които могат да обслужват милиони потребители едновременно всеки ден. В същия времеви период се появяват и специализирани платформи за разработка като ManyChat, Chatfuel и Dialogflow, които значително улесняват създаването и управлението на чатботове предимно за онлайн бизнеси. Съществен пробив при чатбот технологиите се

наблюдава успоредно с развитието на изкуствения интелект и машинното обучение. Чатбот асистенти като Apple Siri, Amazon Alexa, Google Assistant, OpenAI ChatGPT, Google Gemini, Anthropic Claude и др. демонстрират възможността за провеждане на значително по-естествени и непринудени разговори, както и изпълнение на сложни задачи. Вместо те да разчитат единствено на предварително зададени правила като по-старите чатботове, те анализират огромни количества данни с цел самообучение и непрекъснато подобряват своите отговори. В днешно време уеб чатботовете се разглеждат като ключов елемент от цифровата трансформация на бизнеса в дигиталната ера. Те позволяват автоматизация на множество процеси и предоставят нови възможности за взаимодействие между организации и крайни потребители. [3, 5, 6, 7, 8, 9, 10]

2.3 Видове чатботове

1. Чатботове, базирани на правила(rule-based chatbots):

Това са най-простите като сложност чатботове, които работят на принципа на предварително зададени правила и логика като използват откриването на конкретни ключови думи и изрази/фрази, за да върнат релевантен на потребителя отговор. Подходящи са за представяне на стандартна информация и изпълняването на лесни задачи. Няма възможност за адаптация към нови данни, защото всичко е предварително дефинирано, и при по-нестандартна потребителска заявка е възможно логиката им да се срина като хвърли грешка обратно на потребителя.

2. Гласови асистенти:

Те представляват специален вид чатботове, които използват гласово разпознаване и синтез на човешка реч. Най-известните са Siri, Alexa и Google Assistant като те позволяват на потребителите да комуникират с тях чрез изпращане на гласови съобщения и най-

често се използват в мобилни устройства и интелигентни “smart” домашни системи.

3. Чатботове с изкуствен интелект:

AI чатботовете използват машинно обучение и обработка и анализ на естествен език (NLP – Natural Language Processing). За разлика от първите чатботове те могат да разберат контекста на разговора и да интерпретират намеренията на потребителя като накрая предоставят обратно персонализирани и подходящи отговори. С натрупването на нови взаимодействия и данни, върху които да се обучават, те продължават да подобряват динамично своята ефективност.

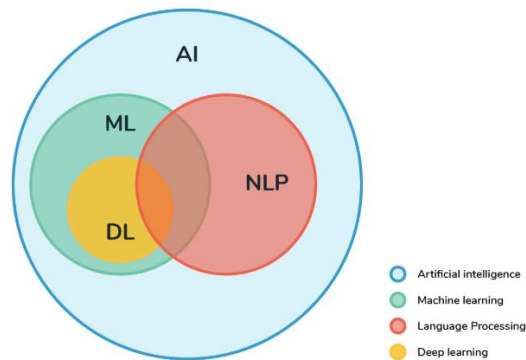
4. Генеративни чатботове с изкуствен интелект:

Най-модерните и съвременни чатботове, които използват големи и сложни езикови модели (LLM – Large Language Models). Те могат да създават свои оригинални текстове, да раздробяват и обобщават информационни потоци, да създават съдържание (напр. изображения) и водят с потребителя сложни и дълбоки разговори. Най-популярните са ChatGPT, Gemini, Claude, Grok, DeepSeek и др. [3, 5, 7, 8, 9, 10]

2.4 Технологиите при съвременните чатботове

Основата, която крепи съвременните чатботове, е изкуственият интелект. Той позволява на системата зад чатбота да анализира потребителските заявки като разбере тяхното значение и определи подходящ отговор. Употребата на изкуствен интелект значително подобрява качеството на провежданите разговори между човек и чатбот

и се доближава до естествена човешка комуникация. Обработката на естествения език е технологията, която позволява на чатботовете да разбират граматиката, значението и контекста в изреченията, въведени от потребителя. Дори могат да игнорират леки правописни грешки и разпознават жаргонни думи, ако общият смисъл и контекст е установен, позволявайки формирането на коректен отговор.

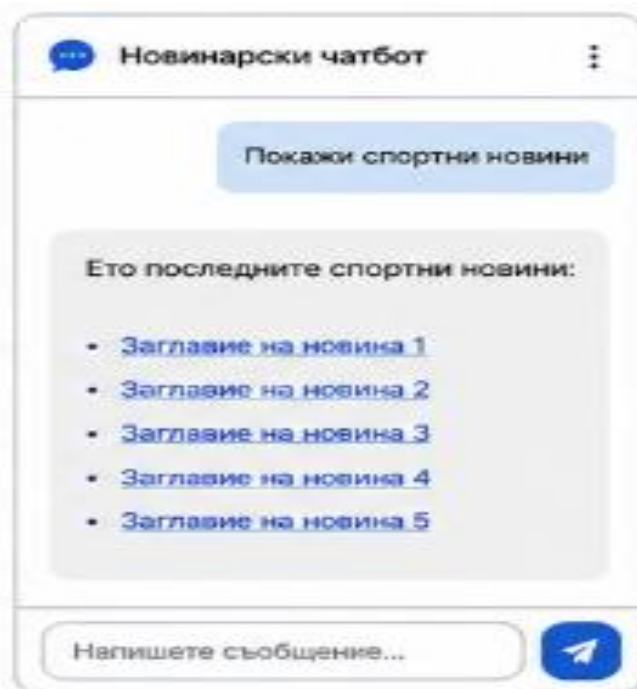


Чрез употребата на машинно обучение чатботовете имат възможността да анализират предишни разговори и постепенно да увеличават качеството на своите резултати. Също така могат да се откриват тенденции и закономерности в потребителите, което дава възможност на чатбота да се адаптира според техните предпочитания. А с помощта на технологиите за разпознаване на намерения (Intent Recognition) могат да се анализират даже и емоции, което позволява още повече персонализация към потребителя. [3, 5, 7, 8, 9, 10, 11]

2.5 Употреба и приложение на чатботовете

Чатботовете намират широко приложение в днешния дигитален свят, но определено една от най-силно разпространените области за тях е обслужването на клиенти. Те могат да предоставят изчерпателна информация за продукти и услуги, да получават и обработват клиентски запитвания, да приемат поръчки и да разрешават често срещани проблеми и затруднения. В електронната търговия чатботът може да

помага на клиента за избор на продукт за покупка, да известява за намаления и промоции, да персонализира препоръките и да съдейства при процеса на покупка. Всичко това подобрява клиентското изживяване и увеличава шанса за продажби на бизнеса. В образователната сфера чатботовете могат да са в услуга на ученици и студенти като отговарят на въпроси, разясняват учебни програми и планове и да подпомагат административните процеси. В сферата на маркетинга чатботовете са способни да изпращат персонализирани предложения на потенциалните клиенти, да събират информация за тях и да помагат на рекламните кампании на компаниите. В новинарските сайтове и медии чатботовете могат да предоставят бърз и лесен достъп до актуална информация, да търсят новини по категории и други ключови думи, подобрявайки взаимодействието на посетителите на сайта и съдържанието в него. [3, 4, 6, 7, 8, 9]



2.6 Предимства и недостатъци на чатботовете

Предимства	Недостатъци
Достъпност 24 часа в денонощието, 7 дни в седмицата, 365 дни в годината	Съществува вероятност за неправилно разбиране на потребителските съобщения
Мигновен отговор на много потребители едновременно в реално време	Затруднения при обработка на сложни или двусмислени въпроси и заявки
Намаляване на разходите за обслужване на клиенти	Ограничена способност за проява на емпатия и човешко отношение
Автоматизиране на рутинни задачи и отговаряне на често задавани въпроси	Нуждае се от периодична поддръжка и актуализация
Осигуряване на последователни и стандартизирани отговори	Качеството зависи от обучаващите данни и конфигурацията на системата

Предимства	Недостатъци
Събиране и анализ на информация за потребителското поведение	Има риск от предоставяне на неточна или непълна/невярна информация
Подобряване на потребителското изживяване чрез мигновено обслужване	Ограничена ефективност при специфични казуси, изискващи човешка намеса
Лесна и бърза интеграция с уебсайтове, социални мрежи и мобилни приложения	Потенциални рискове, свързани със сигурността и защитата на личните данни

Данните от таблицата сочат, че чатботовете осигуряват значителни предимства в аспекта на бързината, достъпността и автоматизацията на стандартната комуникация. Въпреки всичко това те все още не могат напълно да заменят **човешкия фактор** най-вече при решаването на сложни казуси и проблеми, изискващи индивидуален подход и емоционална интелигентност. Заради това много компании и организации предпочитат да използват чатботовете като допълнение към човешките оператори, а не като тяхна пълна замяна. [4, 5, 6, 7, 8, 9]

2.7 Бъдещо развитие и тенденции на чатбот технологиите

Чатботовете отдавна не са просто маркетингов инструмент, а сложна съвременна софтуерна архитектура. Развитието им е тясно свързано с напредъка на изкуствения интелект като се очаква следващите поколения чатботове в близко бъдеще да бъдат още по-интелигентни, по-контекстуално осъзнати и по-способни сами да водят естествени разговори. Все по-голямо значение ще имат сложните генеративни AI модели, които както могат да създават съдържание в реално време, така и могат да решават комплексни задачи. Прогресът при мултимодалните системи ще позволи комбиниране на текст, изображение, звук и видео в обхвата на една комуникационна среда. Прогнозите сочат само и единствено нарастване на внедряването и интеграцията на чатботове в още повече бизнес сектори и обществени и държавни услуги. Чатботове остават като една от най-важните области в дигиталната и интернет комуникация. Комбинирайки възможностите на изкуствения интелект и обработката на естествен език, модерните чатбот технологии бързото, лесно и ефективно обслужване реалност. Тенденциите са приложението на чатботове в най-различни сфери да продължи само да нараства и те да играят все по-важна роля в света на информационните системи. [3, 4, 5, 6, 7, 8, 9, 10]

III. Проектиране на системата

Реализирането на уеб чат бот за новинарски сайт в интернет изисква създаването на **full-stack** уеб приложение с един допълнителен модул – за придобиване и съхраняване на данните като в този контекст данните са самите новини в уебсайта. Най-подходящ за тази цел е чатбот, базиран на предварително зададени правила и параметри (**rule-based chatbot**), който

открива и разпознава в потребителската заявка ключови думи и изрази, на база на които връща подходящ отговор като резултат в потребителския интерфейс. Изборът на сайт е **RazgradNews** с уеб адрес:

<https://razgradnews.net/>

Уебсайтът представлява регионална информационна медия, отразяваща новините в град Разград и областта. В началната страница се съдържат “отличените” новини, които всъщност са **9-те** последно въведени в сайта. Под тях са изобразени някои от основните категории и подкатегории с най-новите новини в съответната категория/подкатегория. В страницата на всяка категория и подкатегория се показват хронологично **последните 10** добавени новини. В header-а и във footer-а на сайта се намират връзки към повечето категории, налични в сайта, както и останалите типични за новинарски сайт линкове за: изпращане на новина, реклама, “за нас”, контакти, условия за ползване, партньори, социални мрежи и др.

За извличане на новините от уебсайта е най-подходящ програмният език **Python** с неговите силни **web scraping** възможности за автоматизирано събиране и обработване на данни от интернет. Основните използвани библиотеки са **Requests** за осъществяване на HTTP заявки към сайта и **BeautifulSoup (bs4)** за анализиране на HTML структурата на страниците и събиране на необходимата информация от тях. За съхранение на новините NoSQL базата данни **MongoDB** е изключително подходяща заради бързото и гъвкаво запазване на новините в **BSON** формат (Binary JSON). Самите новини представляват набор от документи в една колекция в базата данни като за всяка новина са задължителни атрибутите:

Заглавие

Хипервръзка(линк)

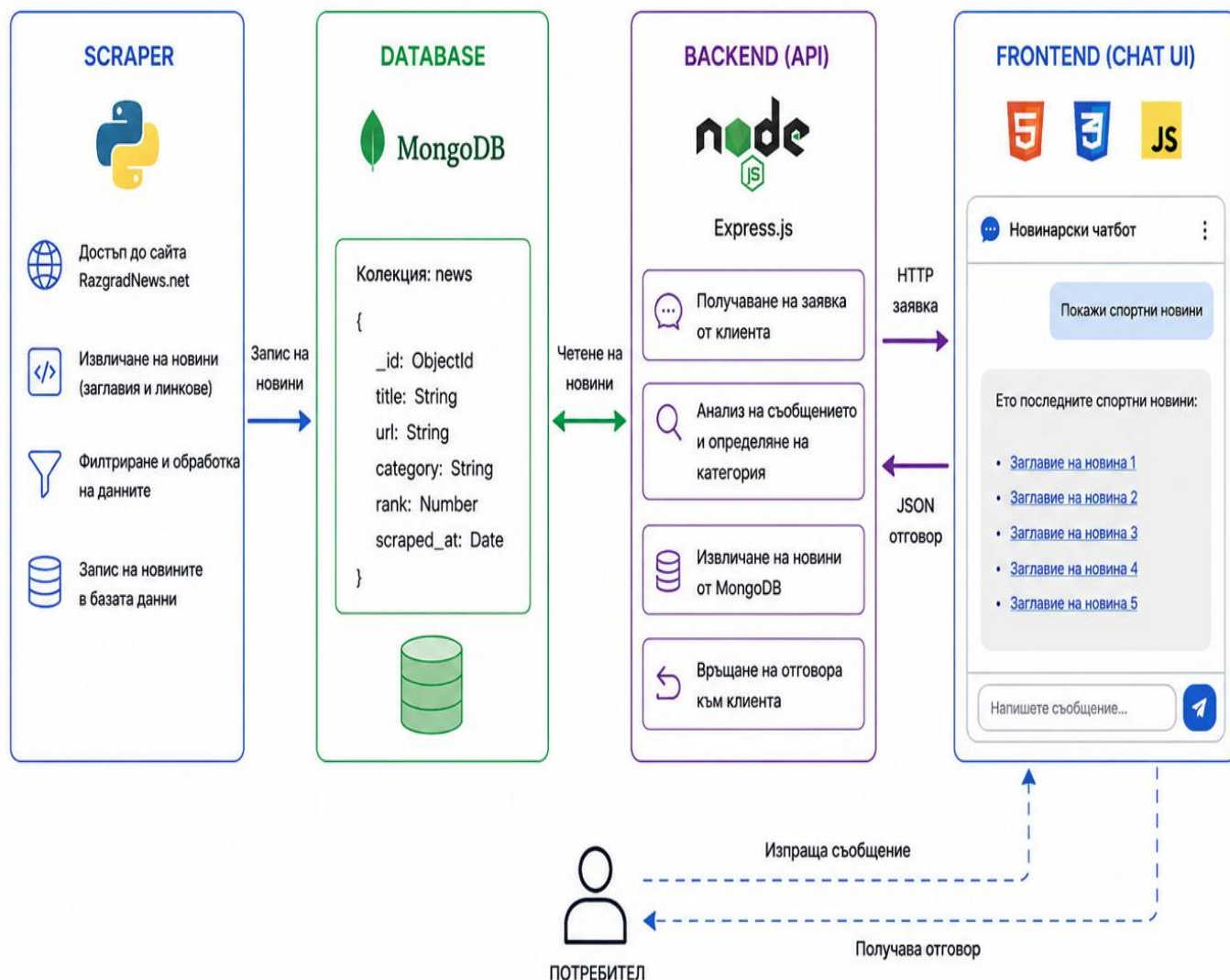
Категория

Връзката между scraping модула и базата данни се осъществява чрез Python библиотеката **PyMongo**.

Backend частта на приложението е захранена от програмната среда **Node.js** с framework **Express.js** – изключително популярно средство за разработка на уеб и мобилни приложения. Backend компонентът се явява като “гръбнак” на системата и медиатор между потребителския интерфейс и базата данни. Ролята му се извява в това да приема потребителски заявки от frontend частта, да ги обработва и да връща подходящ отговор обратно на потребителя. Backend сървърът трябва да предостави API маршрути, чрез които frontend приложението да комуникира със системата, за да се анализира желанието на потребителя като се извлекат релевантните записи от базата данни. Желанието на потребителя към чатбота се състои в изпращане на изречение/команда в **plain text** на български или английски език като backend компонентът има за задача да открие ключови думи в съобщението и да разпознае желаната категория.

Frontend компонентът изгражда потребителския интерфейс на чат бота с помощта на известната комбинация от HTML CSS JS. Той представлява една страница със син преливащ на цвят заден фон и бял прозорец в центъра, който съдържа кликаемо логото на новинарския сайт, поле за писане на съобщение и бутон за изпращане на въведеното съобщение. Работата на frontend-а е да комуникира с backend частта и при получени резултати да визуализира само заглавията на новините с удебелен шрифт като кликаеми линкове към сайта.

Архитектурата на проектираната система е илюстрирана в следната диаграма:

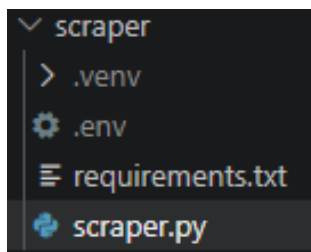


IV. Програмна реализация

Структурата на проекта се състои в 3 основни компонента с едноименните папки: **scraper**, **backend** и **frontend**.

4.1 Scraper

Тук се намират следните файлове:

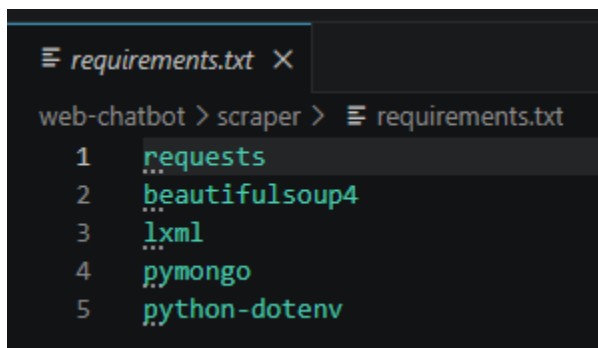


.venv е подпапка, която се създава автоматично при създаване и активиране на виртуална Python среда (обяснено в глава V. Ръководство за използване)

.env е конфигурационен файл, в който се намира **MONGO_URI** – променлива, в която трябва да се запише **ключ** към облачна база данни MongoDB Atlas или ключ за локална MongoDB база данни на локален сървър. Създаването на файла може да бъде пропуснато, когато използваме локална база данни и просто използваме следната стойност по подразбиране на нужното място в програмния код:

<mongodb://localhost:27017/web-chatbot>

requirements.txt – този текстови файл съдържа Python библиотеките, които **scraper.py** използва. Инсталацията им е обяснена в глава V. Ръководство за използване.



Requests позволява изпращането HTTP заявки като GET и POST към уеб сървъри.

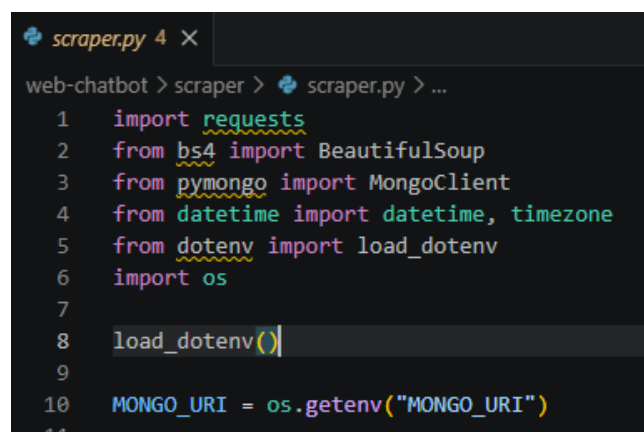
BeautifulSoup4 е подходяща за web scraping, защото успява да вземе HTML структурата на дадена уеб страница. Работи в синхрон с requests.

Lxml – използва се за извличане на данни от XML и HTML файлове.

PyMongo – официален Python драйвер и библиотека за комуникация с **MongoDB** бази данни.

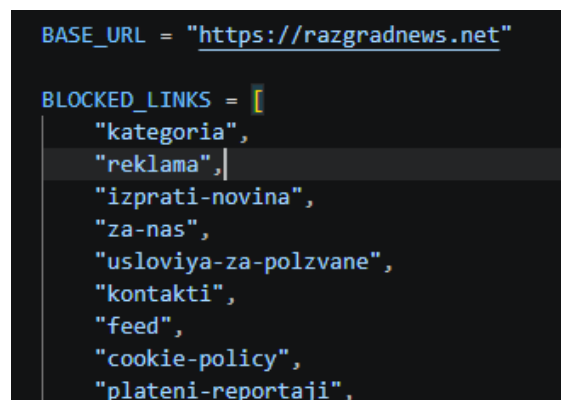
Python-dotenv успява да прочете стойността на ключа **MONGO_URI** от **.env** файла безопасно и да не го разкрива от съображения за сигурност например при публично GitHub repository **.env** файлът бива автоматично поставен в **.gitignore** файла, за да не бъде качвано в интернет съдържанието на **.env**.

Определено най-важният и значим файл в **scraper** модула е **scraper.py**, който съдържа цялата логика за **web scraping** на нужната информация от **razgradnews.net**. Съдържанието на файла започва с внасяне на нужните Python библиотеки и конфигуриране на **MONGO_URI** от **.env** файла.



```
scraper.py 4 X
web-chatbot > scraper > scraper.py > ...
1  import requests
2  from bs4 import BeautifulSoup
3  from pymongo import MongoClient
4  from datetime import datetime, timezone
5  from dotenv import load_dotenv
6  import os
7
8  load_dotenv()
9
10 MONGO_URI = os.getenv("MONGO_URI")
11
```

В константата **BASE_URL** е записан базовият уеб адрес на сайта, за да се използва по-късно за работа с пълни URL адреси. След него е дефиниран списък от нежелани части на URL адреси **BLOCKED_LINKS**, които не представляват новини, и е желателно да се филтрират от логиката по извличане на новини. Такива биват “**reklama**”, “**kontakti**”, “**za-nas**” и др.



```
BASE_URL = "https://razgradnews.net"

BLOCKED_LINKS = [
    "kategoria",
    "reklama",
    "izprati-novina",
    "za-nas",
    "usloviya-za-polzvane",
    "kontakti",
    "feed",
    "cookie-policy",
    "plateni-reportaji",
]
```

След това се намират настройките на **MongoDB** като на последващата екранна снимка е закоментирана локалната база и вместо това се използва **MONGO_URI** от .env файла, в който е записан поверителен ключ към облачна база данни **MongoDB Atlas**, използвана за онлайн хостинг на проекта (използвано за демонстративни цели, справка - github.com/okan-seid/web-chatbot).

```
#Локална база MongoDB
#client = MongoClient("mongodb://localhost:27017/")

#Онлайн база MongoDB Atlas
client = MongoClient(MONGO_URI)

db = client["web-chatbot"]
collection = db["news"]

HEADERS = {
    "User-Agent": "Mozilla/5.0"
}
```

Името на базата данни е **web-chatbot**, а името на колекцията - **news**. Някои сайтове в интернет ограничават автоматизирани HTTP заявки затова е зададен параметър **User-Agent** със стойност "Mozilla/5.0", който симулира истинска заявка от стандартен уеб браузър за коректно и правилно зареждане на съдържанието на уеб страниците.

Последва списък от речници **SOURCES**, в който всеки елемент съдържа двойки: ключ-стойност за името на категорията/подкатегорията и нейния URL адрес в сайта.

```
#Категории и техните url-и в сайта
SOURCES = [
    {
        "url": "https://razgradnews.net/",
        "category": "homepage"
    },
    {
        "url": "https://razgradnews.net/novini-ot-razgrad",
        "category": "razgrad"
    },
]
```

За началната страница е използван базовият адрес на сайта, в който са **9-те последно** добавени новини в сайта. Те се записват в категория **“homepage”**, за да удовлетворяват търсенията за “последни новини”, “актуални новини” и “най-нови новини” в чатбота. За категория “Разград” е записан точният адрес: <https://razgradnews.net/novini-ot-razgrad> и име на категория **“razgrad”** и аналогично по същия начин за всички останали категории/подкатегории. Включените в този списък още са: “Предстоящо”, “Криминални”, “Инциденти”, “Областта”, “България”, “Подкасти”, “Спорт”, “Лудогорец”, “Любопитно”, “Гласът на Разград”, “Свят”, “Завет”, “Исперих”, “Кубрат”, “Лозница”, “Самуил” и “Цар Калоян”. [13]

Съществената логика по извличане на новините се случва във функцията **scrape_source** с параметър **source**, където се подават елементи от по-рано дефинирания списък с речници **SOURCES**. Логично в 2 променливи **url** и **category** се записват съответните стойности.

```
def scrape_source(source):  
    url = source["url"]  
    category = source["category"]
```

С помощта на библиотеката **requests** се изпраща **HTTP GET** заявка към url адреса с дефиниран headers, за да изглежда истинска, и се задава максимално време за изчакване 10 секунди. Полученият HTML код се анализира лесно с **BeautifulSoup** библиотеката и се извличат елементите му.

```
response = requests.get(url, headers=HEADERS, timeout=10)  
soup = BeautifulSoup(response.text, "html.parser")
```

Преди предстоящото записване на новини в базата данни се извършва **изтриване** на старите записи във всяка категория:

```
collection.delete_many({"category": category})
```

Целта е базата данни да съхранява само актуалните 9 новини на показ от началната страница и 10-те най-нови от всяка категория тъй като има по 10 новини на първата страница на всяка категория и подкатегория. Всички

останали новини са на втора страница, трета и т.н. (пример с категория Разград, втора страница - <https://razgradnews.net/novini-ot-razgrad/page/2>). Те не се извличат никога тъй и при всяко **изпълнение** на **scraper.py** чатботът разполага само с **най-скорошните** новини от началната страница и поддържаните категории. В следващия фрагмент програмен код се задават нови променливи за успешно намерени новини, за място в страницата (за да може чатботът да визуализира последователно) и лимит, за да знае кога да спре. **Found** и **rank** са с начални стойности 0, а **limit** е 10 с изключение на началната страница, която има 1 новина с голямо изображение и 8 с малки снимки, т.е. 9 общо. **Seen** представлява множество (set) и се използва с цел предотвратяване на дубликатни записи на новини. [12]

```
found = 0
rank = 0
limit = 10
if category == "homepage":
    limit = 9

seen = set()
```

Следва логика по извличане на самите новини като за началната страница се взимат всички хипервръзки ("a" HTML тагове), а за останалите категории се търсят **h2** и **h3** HTML тагове, които да представляват заглавие на новина. Ако се открият 0 елементи, се обхождат всички връзки отново. За всеки елемент се проверява дали има заглавие и линк след това се извършва проверка дали заглавието е "новини", което няма да представлява истинска новина, а навигационна част от страницата. Аналогично за "author/" се проверява и филтрира и ако условията не се покриват, се прескача елементът чрез операторът **continue**. Проверяват се всички открити линкове дали принадлежат на **BASE_URL**, което е <https://razgradnews.net>, за да се филтрират всички външни линкове (напр. към социални мрежи). Ако се върне непълен линк като например "novini-ot-razgrad", се инкрементира с **BASE_URL** до <https://razgradnews.net/novini-ot-razgrad>, което е реален URL адрес в сайта. В нова променлива **path** се записва само частта от URL адреса на една хипервръзка без **BASE_URL**, което е базовият адрес на сайта. Това се

използва по-късно с цел допълнителна филтрация. Ако **path** случайно е **празен низ**, елементът се пропуска. Наред е допълнителна проверка за линкове, които съществуват в сайта, но не са реални статии/новини.

```
#махаме query параметри, ако има
path = link.replace(BASE_URL + "/", "").strip("/")
path = path.split("?")[0].split("#")[0]

#пропускаме служебни страници и меню категории
if any(path.startswith(blockedLink) for blockedLink in BLOCKED_LINKS):
    continue

#пропускаме категории/подкатегории/архиви без конкретна статия
if path in [
    "novini-ot-razgrad",
    "novini-ot-razgrad/kriminalni-novini-razgrad",
```

Всяка истинска новина/статия има **URL** адрес, сходен на посочения пример:

<https://razgradnews.net/balgariya-s-novo-konsulstvo-v-shtutgart>

Това е новина от края на 2025 година в категория “Свят”, чийто URL адрес ще удовлетвори всички проверки и филтри на алгоритъма, защото има **реален URL адрес**, съдържащ **BASE_URL** в себе си и стойността на **path** би била “**balgariya-s-novo-konsulstvo-v-shtutgart**”, което е валидна стойност за линк на истинска новина. В **BLOCKED_LINKS** са също така линковете на категориите и подкатегиите, но те не представляват истинска новина, т.е. запис, който да искаме да извлечем, затова се филтрират и не се записват. Ако някой потенциален валиден запис е стигнал до този фрагмент код след всички проверки и филтри, се проверява за дубликат, който вече да съществува преди да се запише. Ако линкът е уникален и е налице валиден запис, се записва в множеството **seen** и полето **rank** се инкрементира с 1. В колекцията **news** на базата данни **web-chatbot** се обновява запис/документ с **collection.update_one**, който проверява дали съществува вече предишен запис със същата комбинация от link и category и чрез **\$set** се записват:

1. title – заглавие на новина
2. url – линк на новина
3. category – категория на новина

4. `scraped_at` – дата и час на извличане (административни данни)
5. `rank` – позиция в страницата, има стойност от 1 до 10 (1-9 за категория `homepage`)

```
#уникализиране на линковете за страницата
if link in seen:
    continue
seen.add(link)
#rank по уникални линкове
rank += 1

collection.update_one(
    {"url": link, "category": category},
    {"$set": {
        "title": title,
        "url": link,
        "category": category,
        "scraped_at": datetime.now(timezone.utc),
        "rank": rank
    }},
    upsert=True
)

found += 1
```

upsert=True гарантира създаването на нов запис, ако предишен със същите линк и категория не съществува. По този начин записите в колекцията **news** се или обновяват, или създават. По този начин не се налага проверка за съществуването на документа. Пример за покритие – категория “Разград” е популирана всеки ден с няколко нови новини. Ако се изпълни **scraper.py** веднъж и например 4 астрономически часа по-късно отново, би трябвало да има около 3-7 нови новини за конкретния времеви период. По този начин дадена новина, която е била на второ място в сайта в по-ранния час, а при втория час, който е по-късно през деня, е на 7-мо място например, тя съществува вече в колекцията **news**, но нейните полета имат нужда от обновяване най-вече **rank**, защото има ключова роля при визуализирането на новините в интерфейса на чатбота. Накрая **found** се инкрементира с 1 при успешно намерена и записана в колекцията новина.

```
if limit is not None and rank >= limit:  
    break
```

Следният код осигурява спиране на цикъла по обхождане на елемент чрез **break**, когато лимитът е достигнат (когато rank се изравни с limit).

Максимумът на записани новини в категория "homepage" е 9, а за останалите категории е 10.

```
print(f"{category}: намерени {found} новини")  
  
def run():  
    for source in SOURCES:  
        scrape_source(source)  
  
if __name__ == "__main__":  
    run()
```

print се използва за дебъгване и диагностика и изкарва в конзолата колко реални новини за всяка категория са намерени без значение колко са HTML елементите (може да са над 200-300)

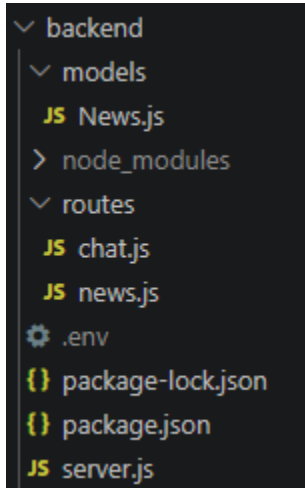
Функцията **run()** управлява изпълнението на scraper модула. Тя последователно обхожда всички елементи в списъка **SOURCES** с подаден параметър **source** на извиканата функция **scrape_source** за всеки елемент source, за да се обходят всички поддържани категории и подкатегории.

```
if __name__ == "__main__":  
    run()
```

Това е стандартна Python конструкция, която гарантира, че при стартиране на файла **scraper.py** чрез командата **python scraper.py**, ще се извика функцията **run()**.

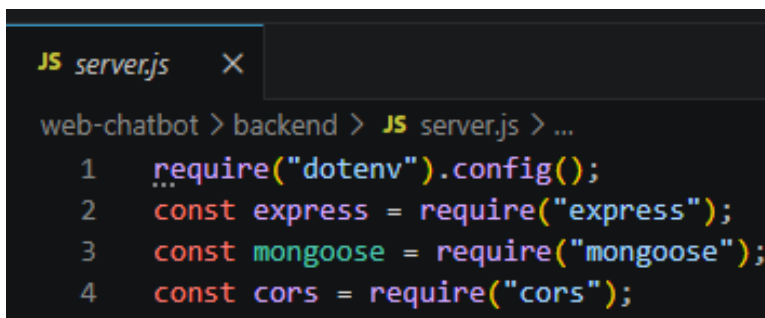
4.2 Backend

Тук се намират следните файлове и подпапки:



Подпапката **node_modules** и двата файла **package.json** и **package-lock.json** се генерират автоматично след изпълняване на **npm install** команда в папка **backend** (обяснено още в глава V. Ръководство за използване) и служат за конфигуриране на **Node.JS** зависимостите.

.env файлът по сходен начин както в scraper модула служи за съхраняване на **MONGO_URI** ключ, но и за променлива **PORT = 3000**. (стойност по подразбиране за сървъри на уеб приложения, използващи **Node.JS** среда).
[14]



Файлът с име **server.js** служи за входна точка на **backend** приложението. В началото на файла се зареждат необходимите библиотеки. **Dotenv** зарежда конфигурационните променливи от **.env**, **express** зарежда **ExpressJS** framework-а, който предоставя функционалността за създаване на уеб сървър

и поставянето на API маршрути, **mongoose** осигурява връзката с **MongoDB** база данни, а **cors** разрешава комуникация между frontend и backend компоненти, когато са разположени на различни домейни.

```
const app = express();  
  
app.use(cors());  
app.use(express.json());
```

Създава се обект **app** от тип **express()** приложение за управление на входящи HTTP заявки и маршрутизация с ъм съответни функции. **Cors()** разрешава обменът на данни между frontend и backend, а **express.json()** позволява автоматичното образуване на JSON заявки към JavaScript обекти. [14, 15]

Следва връзка с MongoDB база данни чрез **mongoose.connect(process.env.MONGO_URI)** като се взима ключът от .env файла. При успешна връзка се изкарва съобщение “MongoDB connected” в конзолата, при грешка се хвърля грешка в конзолата. След това се предоставя на backend сървъра разрешение да използва **2 API** маршрута – файловете **news.js** – за извличане на новини от базата данни и **chat.js** – за логиката на чатбота, в която се обработват потребителските заявки и се генерира отговор, който впоследствие се изпраща на frontend.

```
//Онлайн база MongoDB Atlas  
mongoose.connect(process.env.MONGO_URI)  
//mongoose.connect("mongodb://localhost:27017/web-chatbot")  
  .then(() => console.log("MongoDB connected"))  
  .catch(err => console.error(err));  
  
app.use("/api/news", require("./routes/news"));  
app.use("/api/chat", require("./routes/chat"));
```

В константа **PORT** се зарежда и записва PORT от .env, ако липсва такава, се задава 3000 – стойността по подразбиране.

```

app.get("/", (req, res) => {
  res.send("web-chatbot backend is running");
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

```

Реализиран е **тестов маршрут** с адрес – началната страница на сървъра, за да се гарантира, че backend приложението работи успешно. А с **app.listen** се задава начало на приемането на заявки на конфигурирания порт като се изкарва съобщение в конзолата. Самият backend сървър се стартира с команда **node server.js** в терминал в папка backend както е споменато в [V. Ръководство за използване](#).

Краткият файл **News.js** в директория **backend/models** служи за дефиниране на структурата на документите, съхранявани в MongoDB базата данни. Отново се достъпва MongoDB чрез mongoose и се дефинират полетата на всяка новина и на база на схемата се създава модел с име **News**, който се използва за извършване на операции по четене, запис и актуализация на данни в MongoDB.

```

JS News.js M X
web-chatbot > backend > models > JS News.js > ...
1  const mongoose = require("mongoose");
2
3  const NewsSchema = new mongoose.Schema({
4    title: String,
5    url: String,
6    category: String,
7    rank: Number,
8    scraped_at: Date
9  });
10
11 module.exports = mongoose.model("News", NewsSchema);

```

Файлът **backend/routes/news.js** реализира API маршрута `/api/news`, чрез който frontend приложението получава достъп до новините, записани в базата данни. Тук са реализирани функционалности за извличане на последни новини, търсене по ключова дума и извличане на новини по категория. В началото се зарежда библиотеката **Express** и се създава обект **router** за дефинирането на API маршрути. Зарежда се и моделът **News** за достъп до колекция **news** в базата данни. Чрез **GET** заявка към `/api/news` се връщат последните 10 новини от базата данни като с метода **find()** се извличат самите документи, сортирани по полето **scraped_at** в низходящ ред, тоест най-новите новини да са първи. Полученият резултат се връща обратно в JSON формат.

```
//последни новини
router.get("/", async (req, res) => {
  const news = await News.find()
    .sort({ scraped_at: -1 })
    .limit(10);
  res.json(news);
});
```

Отново чрез **GET** заявка се реализира търсене по ключова дума като търсеният текст се получава с помощта на параметъра **q** от URL адреса.

```
//търсене по ключова дума
router.get("/search", async (req, res) => {
  const q = req.query.q || "";
```

Търсенето се извършва в заглавията на новините чрез регулярни изрази **\$regex**. Параметърът **i** премахва значението на малките и големите букви в израза. Лимитът отново е 10 броя новини и се връщат в JSON масив. В изображения фрагмент код се извличат новините по категория от `/api/news/category/:category`

```

router.get("/category/:category", async (req, res) => {
  try {
    const { category } = req.params;

    const news = await News.find({ category })
      .sort({ rank: 1, scraped_at: -1 })
      .limit(10);

    res.json(news);
  } catch (err) {
    res.status(500).json({ error: "Грешка при зареждане на новини" });
  }
});

```

Търсят се всички новини по дадената категория и се записват в JSON масив, сортирани по позиция както са в сайта. Например най-скорошната новина в категория “Инциденти” е най-отгоре на страницата на категория “Инциденти” и има rank = 1, за да може да се визуализира правилно във frontend-a. Имплементирана е и проверка за грешки, ако заявката се провали, се задава HTTP статус код 500 и съобщения за грешка към клиента. С помощта на “**module.exports = router**” обектът router се експортира, за да може да бъде регистриран и използван в основния файл на backend сървъра – **server.js**.

Най-важният файл за чатбот логиката определено е **chat.js** в backend/routes директорията. Целта му е да обработи потребителското съобщение, да го анализира, да генерира релевантен отговор и да го предостави на frontend частта. Отново в началото се зареждат Express, Router и News както в предния файл. Следва дефиницията на **CATEGORY_MAP**, което е **списък от обекти**, в който за всеки обект се съдържат: набор от ключови думи, име на категорията, пълно име и кратко име за визуализация на категорията. Ключовите думи поддържат български и английски език и представляват за всяка категория/подкатегория – името на категорията/подкатегорията и думи и изрази, представляващи нейни производни, например за категория “Спорт” – спорт, спортни, sport, sports и т.н. Полето category съдържа името на категорията, използвано при заявките към MongoDB. Полето label съдържа пълното наименование на категорията, което се визуализира при отговорите на чатбота като представлява използвана подходяща емотиконка + име на

категория/подкатегория. Полето shortLabel съдържа по-кратко име и се използва при по-кратки съобщения. За категория homepage, което са последните 9 новини в сайта, изобразени в началната страница, има по-голям набор от ключови думи като: скорошни новини, нови новини, актуални новини или просто “покажи новини” това е с цел при търсене на новини в чатбота без конкретна категория, да може да се подкани визуализация на тези новини от началната страница. Дефинира се **POST маршрут /api/chat**, на който frontend приложението може да изпраща потребителските заявки за обработка. В константа **message** се записва съобщението на потребителя от frontend, превръща се само в малки букви и се премахват излишните интервали. В **contextCategory** се записва последно разглеждана категория или NULL, ако няма такава. Това служи за командите “още” и “show more”, за да знае чатботът коя категория новини да показва. В contextOffset се помни колко новини вече са показани – 0, 5 или 10.

```
router.post("/", async (req, res) => {
  try {
    const message = (req.body.message || "").toLowerCase().trim();
    const contextCategory = req.body.contextCategory || null;
    const contextOffset = Number(req.body.contextOffset || 0);
```

Следва проверка за празно съобщение и подкана за въвеждане на съобщение.

```
//Ако няма text input
if (!message) {
  return res.json({
    reply: "Напиши например: „новини от Разград“, „областта“, „предстоящи“ и др."
  });
}
```

Ако потребителят иска още новини от същата категория чрез съответните команди “покажи още”, “show more” и подобни се извършва проверка дали има активна категория на първо място, за да се поискат повече новини, и се подканва за търсене на новини по някоя категория първо.

```
//Ако потребителят иска още новини от последната показана категория
if (
  message === "дай още" ||
  message === "покажи още" ||
  message === "още" ||
  message === "show more" ||
  message === "more"
) {
  if (!contextCategory) {
    return res.json({
      reply: "Няма активна категория. Напиши например: „Покажи Разград/Криминални/Областта...“."
    });
  }
}
```

След това се подготвя заявката за допълнителните новини: `let query = { category: contextCategory }`; и се сортира по `rank` и `scraped_at`.

В `const moreNews` се изпълнява `query` за допълнителни новини в дадена категория, тоест от 6-тата до 10-тата новина в дадена категория. Тук се използва `contextOffset` и се задава пагинация на страницата и лимит от 5. Ако допълнителните новини вече са показани и оставащите са 0, се изкарва съобщение в чатбота, че няма повече новини в съответната категория. В следващия блок програмен код е реализиран отговорът, който се изкарва в чатбота при поискването на допълнителни новини като целта е чатботът да върне съобщението: "Още новини от + `shortLabel`". `ShortLabel` се взима от обекта от `CATEGORY_MAP`, чиято категория съвпада с `contextCategory`, което е категорията в темата на разговора, за която се иска бонус новини.

```
const categoryInfo = CATEGORY_MAP.find(c => c.category === contextCategory);
const categoryName = categoryInfo?.shortLabel || contextCategory;

return res.json({
  reply: `Още новини от ${categoryName}`,
  category: contextCategory,
  nextOffset: contextOffset + moreNews.length,
  news: moreNews.map(n => ({
    title: n.title,
    url: n.url
  })))
});
}
```

Безспорно най-важната част от логиката на чатбота е как той намира категорията, за която потребителят търси новини. Това се получава чрез създаване на нова променлива **matched**, която първоначално няма стойност. Следва **for** цикъл, който обхожда всички “правила”(ключовите думи и изрази), зададени в **CATEGORY_MAP**. Намирането и откриването става чрез **if** проверка метода **some(message.includes())**, където за удовлетворяване на условието за истина, е достатъчно да се открие поне една от ключовите думи. След това **matched** усвоява стойността на цялата категория и цикълът се прекратява, защото вече е налице разпозната категория. При неразпозната категория или съобщение, съдържащо нерелевантна информация (като например “Здравей”), се изпраща подканване за нов опит и примерни имена на категории, които се поддържат.

```
//Намиране категорията по ключови думи
let matched = null;

for (const rule of CATEGORY_MAP) {
  const ok = rule.keys.some(k => message.includes(k));
  if (ok) {
    matched = rule;
    break;
  }
}

//ако не match-не – показваме примерни категории
if (!matched) {
  return res.json({
    reply:
      "Не те разбрах 🤖 Опитай: „последни новини“, „спорт“, „криминални“, „инциденти“, „областта“, „подкасти“."
  });
}
```

След като вече имаме открита категория, е нужно да се подготви сортирана заявка към MongoDB, която да извлече новините. Например при съобщение “покажи спортни новини” **matched.category** ще стане “sport” от по-ранната логика. Единствено за категория “homepage”, което са новините от началната страница”, е зададен лимит от 9 тъй като целта е да се изкарат всичките 9 новини от т.нар. FeaturedGridNews от началната страница на сайта само с едно съобщение. За всички останали категории лимитът е 5. В конзолата се изкарват стойностите на **message** и **matched.category** с цел дебъгване. С **const news = await News.find(query)** се извличат новините от базата данни чрез **query** заявка. Последователно се прилага сортиране, за да се запази визуалният ред както е в сайта, **skip(0)**, защото при първоначално изкарване

на новини за категория, се изкарват първите 5 новини, тоест не се пропускат никакви записи и се прилага лимитът от 5 и 9 само за “homepage”. В последната част от чат логиката се конструира отговорът към backend-а. Нужна е проверка дали изобщо има новини в разпознатата категория, като в случай че няма, се изкарва съобщение, че няма новини в `matched.category`. Подготвя се отговор от **backend -> frontend** в **JSON** формат. Запазва се името на разпознатата категория, самата категория, за да се поддържа във водения разговор с чатбота, пагинация на новините (в случая 5 новини) и чрез метода **map()** се представя опростена структура на новините (само име и заглавие на новината) тъй като в отговорите на чатбота само те се показват. Ако възникне грешка при MongoDB заявката или обработката на отговора, се изкарва грешка в конзолата и в чатбота и системата връща HTTP статус код 500. А с **module.exports = router** се разрешава употребата на `/api/chat` маршрута от основната част на backend компонента – `server.js` файлът.

```
if (news.length === 0) {
  return res.json({ reply: `Няма новини в категория: ${matched.category}`
})

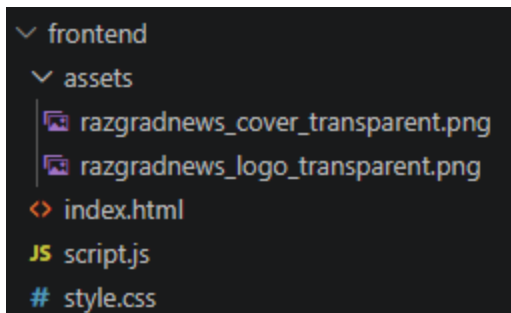
return res.json({
  reply: matched.label,
  category: matched.category,
  nextOffset: news.length,
  news: news.map(n => ({
    title: n.title,
    url: n.url
  }))
});

} catch (err) {
  console.error("Chat error:", err);
  return res.status(500).json({ reply: "Възникна грешка в чата." });
}
});

module.exports = router;
```

4.3 Frontend

Frontend структурата е класическа, състояща се от **index.html**, **script.js** и **style.css** файлове. В подпапка `assets` се намират `.png` снимки на логото и корицата на `razgradnews.net`



Главният файл **index.html** изгражда основната структура на потребителския интерфейс на чатбота. В него са дефинирани заглавната част на страницата, чат контейнера, индикаторът за зареждане на отговора, полето за въвеждане на текст и бутонът за изпращане на написаното съобщение. Файлът е дефиниран като HTML страница с основен език български и кодировка UTF-8 за коректно визуализиране на българските символи. Заглавието на страницата като се отвори в браузър е зададено на: “News Chatbot – RazgradNews” и е свързан **style.css** файлът за стилово оформление. В чат контейнера е зададено име на чатбота и е позиционирано официалното лого на RazgradNews.net, което съдържа хипервръзка за отваряне на самия уебсайт в нов раздел в браузъра за бърза навигация. Елементът **chat-box** се попълва динамично чрез **script.js**. Индикаторът за зареждане на отговор **loading-container** се показва по време на обработване на заявката към backend сървъра. Полето **user-input** позволява на потребителя на чатбота да въведе съобщение, а бутонът до него извиква JavaScript функцията **sendMessage()**, която изпраща потребителското съобщение към backend частта. В края на HTML документа се зарежда **script.js** файлът, който вече реализира потребителската бизнес логика на frontend частта от приложението.

В **script.js** е реализирана бизнес логиката на frontend частта на чатбота. Файлът започва с декларирането на идентични на backend-а променливи за запомнената категория и вече показаните новини като началните им стойности са 0. Избира се HTML елементът “**chat-box**” и в него чрез функцията **addMessage()** се добавя нов HTML елемент за всяко съобщение, както от

потребителя, така и от чатбота. Параметърът **sender** определя визуалната позиция на съобщението в чат кутията според това чие е съобщението. Чатботът автоматично превърта прозореца до най-новото съобщение чрез **chatBox.scrollTop = chatBox.scrollHeight**. След това се избират HTML елементите, отговорни за индикатора за зареждане и се използват само докато frontend частта чака отговор от backend частта на приложението. Дефинира се асинхронна функция **sendMessage()** (която е и свързана с бутона за изпращане), която изпраща HTTP заявки към backend API сървър и изчаква отговор. Функцията в същността си взема въведения от потребителя текст и го присвоява в променлива **message** след като самият **input** бива взет и приложен **trim()** на съобщението за премахване на излишните интервали в началото и края на въведения текст. Извършва се проверка дали потребителят изобщо е въвел някакъв текст преди да се изпрати заявка към сървър и ако е въвел, съобщението му се добавя в чат контейнера с параметър **sender=user** и **input** полето се изчиства за следващото съобщение. Напредъкът на индикатора за зареждане започва от 0 и показва стойността си до 90%, когато спира, докато frontend частта чака отговор от backend сървър.

```
const chatBox = document.getElementById("chat-box");

function addMessage(html, sender) {
  const div = document.createElement("div");
  div.className = `message ${sender}`;
  div.innerHTML = html;
  chatBox.appendChild(div);
  chatBox.scrollTop = chatBox.scrollHeight;
}

const loadingContainer = document.getElementById("loading-container");
const loadingBar = document.getElementById("loading-bar");

async function sendMessage() {
  const input = document.getElementById("user-input");
  const message = input.value.trim();

  if (!message) return;

  addMessage(message, "user");
  input.value = "";

  loadingContainer.style.display = "block";
  loadingBar.style.width = "0%";
}
```

Отговорът на backend-а се присвоява в променлива **response**, взета от заявка с **await fetch(<http://localhost:3000/api/chat>)**, на който адрес локално на порт 3000 се помещава API маршрутът на **api/chat.js** от backend компонента.

Методът е POST, а данните от заявката се конвертират в JSON формат. После полученият от backend JSON отговор се преобразува в JavaScript обект:

```
const data = await response.json();
```

Индикаторът за зареждане завършва своя прогрес от 90% на 100% и след кратко време (0.3 секунди) се скрива като намаля своята ширина до 0, ефективно изчезвайки от визуалното пространство. Ако backend отговорът върне категория и стойност на отместване в отговора, съответните променливи се запазват в по-рано дефинираните lastCategory и lastOffset, за да може механизмът за допълнителни новини да работи правилно.

```
if (data.category) {  
  lastCategory = data.category;  
}  
  
if (typeof data.nextOffset === "number") {  
  lastOffset = data.nextOffset;  
}
```

Полученият отговор се проверява дали съдържа масив с новини и ако съдържа, се създава HTML структура със заглавие и списък от новини. Всяка новина, която е HTML <listitem> в този списък, се визуализира като кликаемо заглавие, съдържащо линк към новината в сайта. Атрибутът rel="noopener noreferrer" повишава сигурността при отваряне на външни връзки в браузър.

```
if (Array.isArray(data.news) && data.news.length > 0) {  
  let html = `<div class="news-header"><strong>${data.reply || "Резултати"}</strong></div>`;   
  html += `<ul>`;   
  
  data.news.forEach((n) => {  
    html += `<li><a href="${n.url}" target="_blank" rel="noopener noreferrer">${n.title}</a></li>`;   
  });  
  
  html += `</ul>`;   
  
  addMessage(html, "bot");  
  return;  
}
```

Накрая изкараното съобщение в чатбота отново се добавя като нов HTML елемент в чат контейнера, но този път параметърът sender=bot. Ако няма никакви новини за показване, е необходимо да се обработи съобщението за

липса на резултати чрез обработване на специалните HTML символи < и >, за да се предотврати изпълнението на нежелан/злонамерен HTML код от потребителския интерфейс.

```
const safeReply = (data.reply || "Няма резултати.")
  .toString()
  .replace(/</g, "&lt;");
  .replace(/>/g, "&gt;");

addMessage(`<strong>${safeReply}</strong>`, "bot");
}
```

Съобщението накрая се добавя в чат контейнера като отново sender = bot.

Освен натискането на бутона за изпращане е имплементиран и **EventListener** за кликане на бутона **Enter** от клавиатурата, при което отново се извиква функцията **sendMessage()**.

Накрая е добавен **EventListener** на ниво window, т.е. се включва още при

```
document.getElementById("user-
input").addEventListener("keydown", (e) => {
  if (e.key === "Enter") sendMessage();
}):
```

отваряне на **index.html** в браузъра. Той проверява дали чат контейнерът е празен, и ако е изпраща приветстващо съобщение, съдържащо кратко представяне на чатбота, подкана към търсене на новини на български или английски език, списък с част от поддържаните категории и командите за допълнителни новини – “още”, “дай още”, “more” и др.

```

(chatBox.children.length === 0) {
addMessage(
  "👋 <strong>Добре дошли в web чат бота на RazgradNews!</strong><br/>" +

  "Търсете актуални новини по категории на български или английски език.<br/><br/>" +

  "<strong>Поддържани категории:</strong><br/>" +
  "Нови/Последни/Актуални<br/>" +
  "Разград/Предстоящо/Криминални/Инциденти<br/>" +
  "Спорт/Областта/България/Свят<br/>" +
  "Любопитно/Подкасти/Гласът на Разград<br/><br/>" +

  "<strong>Примерни команди:</strong><br/>" +
  "• Покажи новини/актуални/последни/нови<br/>" +
  "• Покажи новини за Разград<br/>" +
  "• Покажи спортни новини<br/>" +
  "• Show world<br/>" +
  "• Show podcasts<br/>" +
  "• Още / дай още / more / show more",
  "bot",
  true
)
}

```

Съобщението бива добавено чрез addMessage() с параметър sender = bot. Това приветстващо съобщение винаги ще се изкарва при опресняване на страницата и при зареждане на index.html отначало, защото в началото винаги чат контейнерът е празен, и if проверката в EventListener-а ще бъде със стойност истина.

Оформлението на стиловете и визуалната естетика на HTML документа са реализирани във файла style.css. Дефинирани са цветовата схема, размерите и позиционирането на отделните HTML елементи, стиловете на съобщенията, полето за въвеждане на текст, бутонът за изпращане и индикаторът за зареждане. Използван е интерфейс с преливащ на цвят заден фон, ясно разграничаване на съобщенията на потребителя и на чатбота и лесни за употреба текстово поле и бутон за изпращане на заявка. Основната цел е подобряване на качеството на потребителското изживяване чрез CSS стилизация за постигане на модерен, интуитивен и удобен за ползване новинарски чатбот, който позволява бърз и лесен достъп до новините в сайта.

V. Ръководство за използване

Необходима инсталация на инструменти и софтуер за изпълнение на приложението:

1. Microsoft Visual Studio Code (основна среда и IDE)
2. Python (използван за изпълнение на **scraper.py** файла)
3. Node.js (опцията **Add to PATH** задължително включена, използван за backend език и Express.js Server)
4. MongoDB Community Server (сървърна инфраструктура за MongoDB база данни)
5. MongoDB Compass (графичен интерфейс за управление на MongoDB база данни)
6. Git (ПО ИЗБОР – за source version control)
7. Съвременен уеб браузър (например Google Chrome)

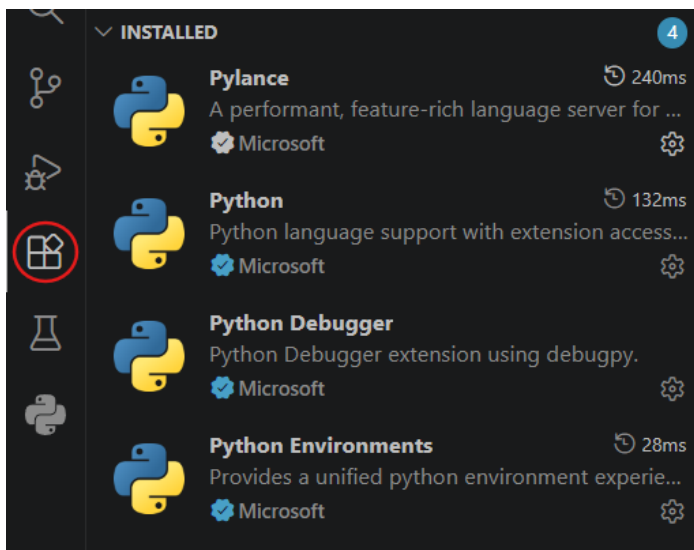
Сдобиването със source code-а на проекта става чрез Git Clone или Clone Repository на web repository-то, намиращо се на уеб адрес:

<https://github.com/okan-seid/web-chatbot>

или

чрез copy-paste локално на папка **web-chatbot** от предоставения електронен носител (USB флашка)

След като проектът е вече наличен локално и отворен във VSCode, е препоръчително да се инсталира пакетът от най-популярните Python разширения.



Необходимо е създаване на Python Virtual Environment за scraper модула. Отваря се Integrated Terminal в **web-chatbot/scraper** и се изпълняват последователно следните Python команди:

```
python -m venv .venv  
.\.venv\Scripts\Activate.ps1  
pip install -r requirements.txt
```

По този начин се създава и активира виртуална среда и се инсталират нужните Python библиотеки, описани в **requirements.txt** файла. При **втората** команда е възможно да изкара грешка за блокирано изпълнение на локални скриптове. Отстранява се чрез отваряне на Windows PowerShell (като администратор) и изпълняването на команда: **Set-ExecutionPolicy -Scope CurrentUser RemoteSigned** и после **Y** за даване на съгласие.

Настройването на **backend** средата и инсталацията на **Node.js** зависимостите се извършва чрез отварянето на терминал в **web-chatbot/backend** и изпълняването на последващата команда: **npm install**

Впоследствие трябва да се създаде **.env** файл в същата директория на backend-а и да се конфигурира със следните настройки:

(Стойност по подразбиране за **MongoDB** база данни с име **web-chatbot**)

MONGO_URI=mongodb://localhost:27017/web-chatbot
PORT=3000

[13]

При успешна и правилна инсталация трябва във фонов режим в Windows да се изпълнява следният процес:

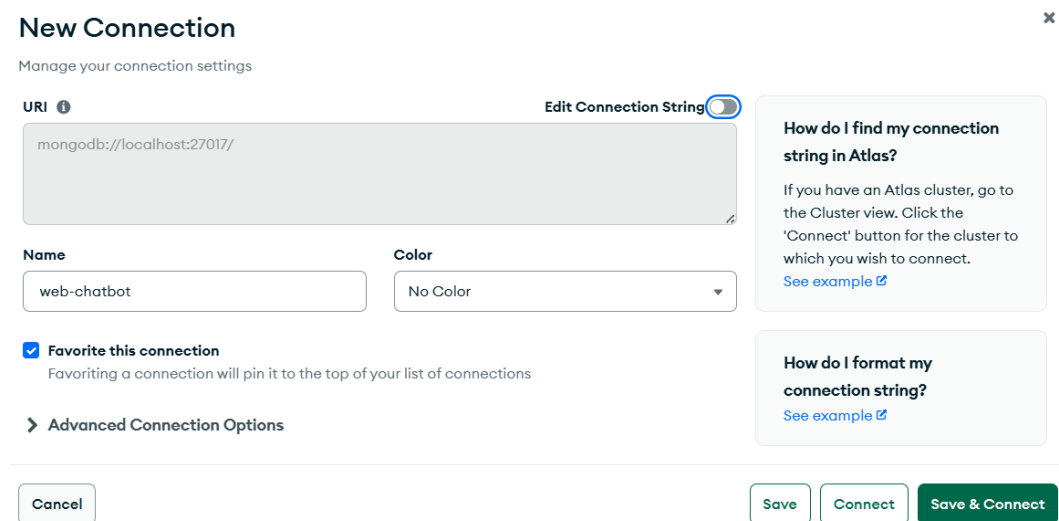
	Microsoft OneDriveFile Co-Au...	0%	7,9 МБ	0 МБ/с	0 Мбит/с	
>		Microsoft Windows Search Ind...	0%	9,0 МБ	0 МБ/с	0 Мбит/с
>		MongoDB Database Server	0,1%	36,1 МБ	0,1 МБ/с	0 Мбит/с
	MoUSO Core Worker Process	0%	16,1 МБ	0 МБ/с	0 Мбит/с	
	Realtek HD Audio Universal S...	0%	2,8 МБ	0 МБ/с	0 Мбит/с	

В случай че не се изпълнява, трябва да се стартира от Services ->

MongoDB Server (MongoDB) -> Start

	Microsoft Windows SMS Router S...	Routes mess...	Manual (Trigger Start)	Service
	Mobile Connectivity Managemen...	This service ...	Manual (Trigg...	NT SERVICE...
	MongoDB Server (MongoDB)	MongoDB D... Running	Automatic	Network Se...
	MTKBTSVC	Running	Automatic	Local System
	Natural Authentication	Signal aggre...	Manual (Trigg...	Local System
	Net.Tcp Port Sharing Service	Provides abil...	Disabled	Local Service

След това се отваря **MongoDB Compass** и се създава нова връзка със стойностите по подразбиране



New Connection ✕

Manage your connection settings

URI ⓘ Edit Connection String 

mongodb://localhost:27017/

Name: web-chatbot Color: No Color

☒ **Favorite this connection**
Favoriting a connection will pin it to the top of your list of connections

➤ Advanced Connection Options

How do I find my connection string in Atlas?
If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.
[See example](#)

How do I format my connection string?
[See example](#)

Cancel Save Connect Save & Connect

(името няма значение тъй като това е името на връзката, не на самата базата данни)

Отново е нужна навигация до web-chatbot/scrapper/**scraper.py** файла, за да се свери следният фрагмент код:

```
BASE_URL = "https://razgradnews.net"

#Локална база MongoDB
client = MongoClient("mongodb://localhost:27017/")

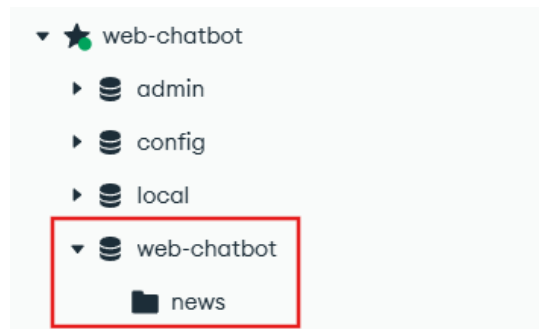
#Онлайн база MongoDB Atlas
#client = MongoClient(MONGO_URI)
db = client["web-chatbot"]
collection = db["news"]
```

Трябва да е активен **само** редът код, указващ връзката с **локален MongoDB клиент**.

Следва да се пристъпи към изпълняване на scraper модула като за целта отново се отваря терминал в папка **scraper** и е необходимо да се изчака кратък период от време(2-3 секунди), за да се активира **виртуалната Python среда** и се подава командата:

```
python scraper.py
```

Автоматично се създава база данни **web-chatbot** с колекция **news** и се запълва с документи, които представляват новините, взети от уебсайта в реално време. С помощта на **MongoDB Compass** може ръчно да се провери съдържанието на колекцията в базата данни.

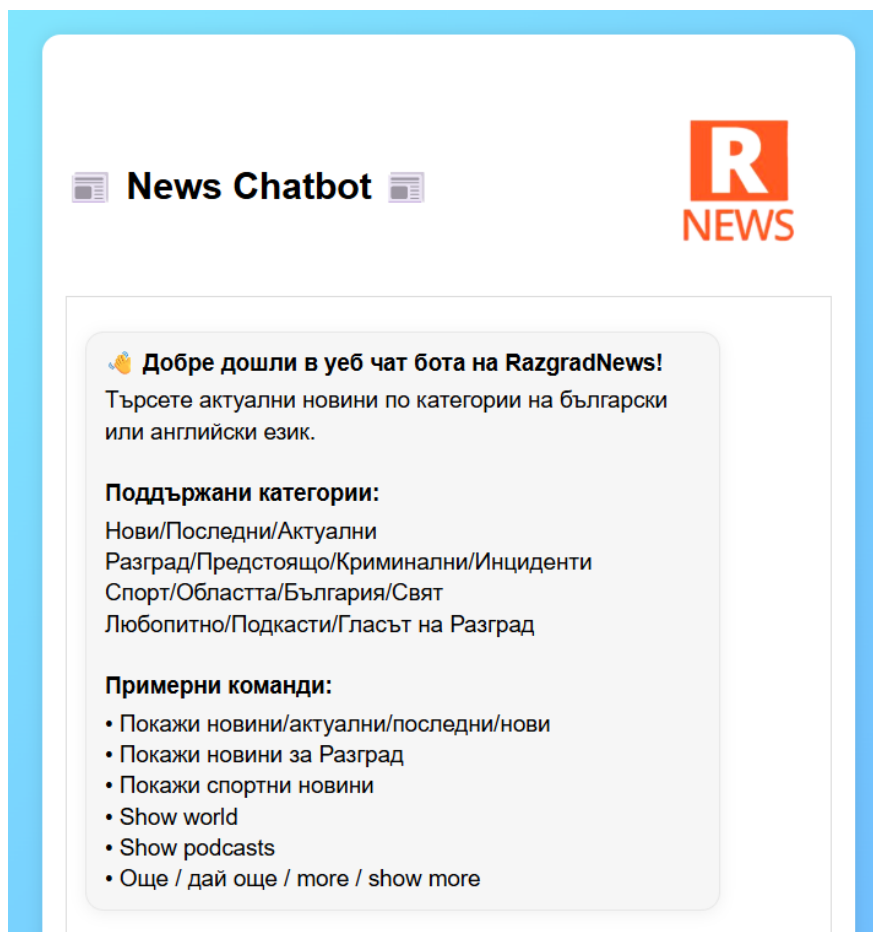


Пускането на **backend** сървъра става чрез отваряне на терминал в **web-chatbot/backend** и изпълняване на:

```
node server.js
```

```
p\web-chatbot\scraper\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\Okan\Desktop\web-chatbot\backend> node server.js
[dotenv@17.2.3] injecting env (2) from .env -- tip: 🚀 run anywhere w
Server running on http://localhost:3000
MongoDB connected
```

Backend сървърът е пуснат и може да взаимодейства с базата данни и frontend-а. За визуализация на чатбот приложението се отваря файлът **frontend/index.html** в уеб браузъра, който изглежда по следния начин:



Използва се предоставеният **графичен интерфейс** (поле за въвеждане на съобщение и бутон за изпращане) и се сверява с уебсайта **razgradnews.net** в реално време за коректност на показаните резултати.

VI. Заключение

Постигната е успешна уеб чат бот функционалност за новинарския сайт RazgradNews.net, която дава възможност за научаване на заглавията на актуалните новини за Разград и региона без нужда от посещаване на уебсайта. Реализацията е извършена по остарели методологии на откриване на ключови думи и изрази във въведеното от потребителя съобщение към чатбота. Позитивите са, че по този начин чатботът никога няма да дава грешна информация или да “халюцинира” и е подходящ за по-опростени чатбот нужди и изисквания като такива на един новинарски сайт. Минусите са , че

работоспособността на системата е податлива на правописни грешки или неточност/неяснота в потребителската заявка, има ограничена поддръжка на наличните категории в сайта, съществува възможност за хронологична неточност поради нуждата от ръчно изпълнение на scraper модула и е налице липсата на извличане и съхранение на по-стари новини по категории в архив. Чатботът може да бъде подобрен и осъвременен с имплементацията на разпознаване на естествен човешки език, използването на машинно обучение и изкуствен интелект и автоматизирането на процеса по извличане на новини (web scraping) в реално време при всяка употреба на чатбота, за да бъде винаги точен и актуален.

VII. Използвани източници

1. https://bg.wikipedia.org/wiki/Уеб_чат
2. <https://bg.wikipedia.org/wiki/Чатбот>
3. <https://postvai.com/seo/chat-botove.html>
4. <https://www.nit.bg/services/izrabotvane-na-chat-bot-za-vashiyat-uebsajt>
5. <https://digital.softuni.bg/blog/what-is-a-chatbot>
6. <https://www.infoserv.bg/web-chat-vs-chatbot>
7. <https://chatbotbg.com/kak-rabotyat-chatbotovete>
8. <https://seomax.bg/kakvo-e-chatbot-i-kak-raboti>
9. <https://www.ibm.com/think/topics/chatbots>
10. <https://www.ibm.com/think/topics/chatbot-types>
11. <https://netpeak.bg/blog/nlp-ili-obrabotka-na-estestveniya-ezik-sashtnost-harakteristiki-i-vazmozhnosti>
12. <https://www.python.org/doc>
13. <https://www.mongodb.com/docs>
14. <https://nodejs.org/docs/latest/api>
15. <https://expressjs.com/en>
16. HTML CSS JS - <https://developer.mozilla.org/en-US>